PGDM v2 — Deterministic Card Dealing Protocol (Continuation)

---

1. Objetivo del PGDM v2

PGDM v2 (Protocol for Game Deck Management v2) define el sistema de generación, barajado y distribución de cartas como un **proceso completamente determinístico, verificable y reproducible**, eliminando cualquier dependencia de RNG opaco o entidades centralizadas.

El objetivo formal es:

Garantizar que cualquier mano de poker pueda ser reconstruida bit a bit por cualquier nodo auditor a partir de los mismos inputs criptográficos.

---

2. Principios fundamentales

PGDM v2 se basa en cuatro principios invariantes:

2.1 Determinismo absoluto

Mismos inputs → mismo deck siempre.

f(seed, state, commits_players) → deck único

2.2 No-RNG externo

No existe fuente aleatoria externa oculta.

- No hardware RNG
- No entropy server
- No seed privado del operador

2.3 Dependencia de estado global

El deck depende del estado del sistema:

- mano actual
- historial de commits
- hash de bloque anterior

2.4 Reproducibilidad universal

Cualquier nodo puede reconstruir:

- orden de cartas
- distribución exacta
- historial de reparto

---

3. Construcción del seed global

El seed no es un valor único, sino una composición criptográfica:

```
Seed_global = H(
   State_previous_hash
   + Commit_Player_Set
   + Time_window_nonce
   + Node_consensus_hash
)
```

Donde:

- `H` = función hash criptográfica (SHA-256 o equivalente)
- `Commit_Player_Set` = todos los commits de acción previos a distribución
- `Node_consensus_hash` = resultado de consenso entre nodos anfitriones

---

4. Generación del mazo

4.1 Mazo base

Se define el mazo estándar:

- 52 cartas
- sin comodines

Representación:

Deck_0 = [A♠, 2♠, ..., K♦]

4.2 Transformación determinística

El mazo se transforma mediante permutación criptográfica:

Deck_t = Permute(Deck_0, Seed_global)

4.3 Algoritmo de permutación

El algoritmo opera como:

1. Expandir seed en stream pseudo-determinístico
2. Mapear bytes → índices de swap
3. Ejecutar Fisher–Yates determinístico

```
for i from n-1 to 1:
   j = H(seed || i) mod (i+1)
   swap(deck[i], deck[j])
```

---

5. Propiedad crítica del sistema

🔴 En sistemas tradicionales:

El shuffle es irreproducible.

🟢 En PGDM v2:

El shuffle es una función matemática fija.

---

6. Distribución de cartas

6.1 Orden de distribución

Las cartas se asignan en orden secuencial del deck permutado:

Player_k receives deck[position_k]

6.2 Regla de consistencia

Todos los nodos deben derivar:

- mismo deck
- misma asignación
- mismo orden de acción

---

7. Validación entre nodos

Cada nodo auditor ejecuta:

7.1 Reproducción local

Deck_rebuilt = PGDM_v2(State_snapshot)

7.2 Comparación

```
if Deck_rebuilt == Deck_consensus:
   VALID
else:
   REJECT_STATE
```

---

8. Prevención de manipulación

PGDM v2 elimina vectores clásicos:

Vector de ataque	Mitigación
Dealer manipulation	Eliminado (no RNG central)
Seed bias	Seed depende de consenso
Replay fraud	Hash chain impide modificación

Hidden shuffle	Permutación pública reproducible

---

9. Relación con LCCP (Crack Chain Core)

PGDM v2 no es un sistema aislado:

- Se ejecuta dentro del **ciclo de mano del protocolo LCCP**
- Cada deck es un “sub-estado verificable”
- Cada mano es un bloque lógico dentro de la cadena de eventos

---

10. Propiedad emergente del protocolo

PGDM v2 convierte el poker en:

Un sistema de distribución de información determinística bajo consenso distribuido.

Esto implica:

- no hay “suerte oculta”
- solo dependencia de estado inicial y acciones humanas
- el resultado es computacionalmente derivado, no generado

---

11. Estado del diseño PGDM v2

El protocolo queda definido como:

```
PGDM_v2 = {
   seed_generation,
   deterministic_shuffle,
   sequential_deal,
   cross_node_validation
}
```

➜ Siguiente bloque lógico

El siguiente nivel técnico natural es:

“LCCP Hand Lifecycle Protocol”

LCCP Hand Lifecycle Protocol

(Crack Chain Poker — Execution Layer Specification)

---

1. Definición formal

El **LCCP Hand Lifecycle Protocol (HLCP)** define el ciclo completo de ejecución de una mano de poker dentro de la red Crack Chain, desde su creación hasta su liquidación final en estado distribuido.

Se modela como una **máquina de estados distribuida determinística**, ejecutada sobre nodos anfitriones con validación cruzada.

---

2. Modelo general

Una mano es un proceso finito:

Hand = { S0 → S1 → S2 → ... → Sf }

Donde:

- `S0` = creación de mesa / inicialización
- `Sf` = liquidación final del estado (pot distribuido + cierre hash)

---

3. Estados del ciclo de mano

3.1 STATE 0 — INITIALIZATION

Función:

Crear instancia de mesa determinística.

Entrada:

- Lista de jugadores (wallet-based IDs)
- Nodos anfitriones activos (n = 40 estándar)
- Parámetros de mesa (blinds, formato, reglas)

Salida:

Table_State_0 = Hash(config + node_set + time_window)

3.2 STATE 1 — SEED SYNCHRONIZATION

Función:

Sincronizar entropía base del sistema.

Proceso:

- Cada nodo genera contribución parcial de seed
- Se realiza consenso de seed global

Seed_global = Consensus(Seed_node_1 ... Seed_node_n)

3.3 STATE 2 — DECK GENERATION (PGDM v2)

Función:

Generación del mazo determinístico.

- Se ejecuta PGDM v2
- El deck es idéntico en todos los nodos

Salida:

Deck_t = PGDM_v2(Seed_global, State_0)

3.4 STATE 3 — DEALING PHASE

Función:

Distribución secuencial de cartas.

Reglas:

- Distribución round-robin determinística
- Cada acción es un evento firmado

Player_i → Card_j = Deck_t[position]

3.5 STATE 4 — ACTION LOOP (CORE GAME LOOP)

Función:

Ejecución de rondas de apuestas.

Cada acción es un evento:

```
Action_t = {
   player_id,
   type (fold/call/raise/check),
   amount,
   signature,
   timestamp_hash
}
```

Validación por nodo:

Cada nodo verifica:

- firma válida
- secuencia coherente
- saldo suficiente
- transición permitida

Transición de estado:

S_t+1 = Transition(S_t, Action_t)

3.6 STATE 5 — CONSENSUS VALIDATION LOOP

Función:

Sincronizar estado entre nodos anfitriones.

Regla:

Valid(S_t) = majority(hash(S_t)) >= 2/3 nodes

Si falla:

- rollback al último estado válido
- aislamiento de nodo divergente

3.7 STATE 6 — SHOWDOWN (IF APPLICABLE)

Función:

Evaluación final de manos activas.

Proceso:

- reconstrucción determinística de combinaciones
- evaluación idéntica en todos los nodos

Winner = f(hands, Deck_t, ruleset)

3.8 STATE 7 — POT SETTLEMENT

Función:

Distribución del pot.

Regla:

- cálculo determinístico del ganador
- validación cruzada de payout

Balance_update = deterministic_distribution(Winner)

3.9 STATE 8 — FINALIZATION

Función:

Cerrar la mano como bloque verificable.

Output final:

```
Hand_Block = Hash(
   S0 → Sf,
   Deck_t,
   Action_log,
   Consensus_log
)
```

Este bloque se agrega al ledger del sistema.

---

4. Propiedades del ciclo HLCP

4.1 Inmutabilidad lógica

Una vez finalizada la mano:

- no puede alterarse el historial
- cualquier cambio rompe el hash chain

4.2 Reproducibilidad total

Cualquier nodo puede reconstruir:

- deck completo
- acciones
- resultado final

4.3 Independencia de servidor central

No existe:

- dealer central
- RNG central
- autoridad única de resolución

---

5. Relación con PGDM v2

HLCP depende directamente de:

- PGDM v2 → generación del deck
- Seed global → entropía inicial
- Consensus layer → validación de estado

Relación estructural:

HLCP = Execution_Layer(PGDM_v2 + LCCP_Consensus + State_Machine)

---

6. Propiedad emergente del sistema

El sistema completo se comporta como:

Una red de ejecución determinística de juegos con verificación distribuida en tiempo real.

Esto implica:

- cada mano es un “bloque lógico”
- cada acción es una transición verificable
- el resultado es matemáticamente reconstruible

---

7. Estado final del protocolo

HLCP queda definido como:

```
HLCP = {
  initialization,
  seed_sync,
  pgdm_deck_generation,
  dealing,
  action_loop,
  consensus_validation,
  showdown,
  settlement,
  finalization
```

}

➜ Siguiente nivel lógico

El siguiente módulo natural del sistema Crack Chain es:

“LCCP Node Host Incentive & Operational Model”

LCCP Node Host Incentive & Operational Model

(Crack Chain Poker — Economic + Operational Layer Specification)

---

1. Definición formal

El **LCCP Node Host Incentive & Operational Model (NHIO Model)** define cómo los **nodos anfitriones (host nodes)** sostienen, operan y son incentivados dentro de la red Crack Chain Poker.

Este modelo conecta:

- operación técnica del sistema
- sostenibilidad económica de la red
- distribución de carga de ejecución
- estabilidad del consenso distribuido

---

2. Principio base del modelo

El sistema se construye bajo una restricción estructural:

La red debe ser autosostenible sin depender de un servidor central ni subsidios externos.

Formalmente:

Network_Operational_Cost ≤ Network_Fees_Generated

---

3. Estructura de nodos anfitriones

Se define un conjunto estándar:

N = 40 Host Nodes (standard cluster)

Cada nodo cumple funciones simultáneas:

3.1 Función operativa

- ejecución de estado de juego
- validación de acciones
- participación en consenso

3.2 Función de infraestructura

- procesamiento de manos
- replicación de estado
- verificación de PGDM v2

3.3 Función económica

- recibe compensación proporcional a carga

---

4. Modelo de carga operativa

Cada nodo tiene una variable de carga:

L_node = f(hands_processed, validation_ops, consensus_participation)

Donde:

- mayor tráfico → mayor carga
- mayor carga → mayor recompensa proporcional

---

5. Modelo de incentivos

5.1 Unidad económica del sistema

La unidad base es:

Fee por mano jugada (Hand Execution Fee)

5.2 Distribución del fee

Cada mano genera un fee total:

F_total = rake_per_hand + operational_fee

Distribución:

```
F_total → {
   Host_nodes (70–85%),
   Network_reserve (10–20%),
   Audit_layer (5–10%)
}
```

---

6. Reparto entre nodos

Cada nodo recibe proporcionalmente:

Reward_i = F_host * (L_i / Σ L_all_nodes)

Donde:

- `L_i` = carga del nodo i
- distribución dinámica basada en participación real

---

7. Incentivo de comportamiento correcto

El modelo penaliza comportamiento no válido:

7.1 Penalización

- nodos con desincronización
- nodos que fallan validación

- nodos con inconsistencias de hash

Reward_i = Reward_i - penalty_factor

7.2 Exclusión del consenso

Si un nodo diverge repetidamente:

- es removido del conjunto activo
- pierde participación en distribución de fees

---

8. Modelo operativo del sistema

El sistema funciona como una **red de ejecución distribuida remunerada por actividad real**.

Flujo operativo:

1. jugador ejecuta acción
2. nodo procesa evento
3. consenso valida
4. sistema registra transición
5. fee se genera
6. nodos reciben recompensa proporcional

---

9. Balance económico del sistema

Condición de estabilidad:

Σ Rewards_nodes + Reserve + Audit_cost ≤ Total_fees_generated

Esto garantiza:

- sostenibilidad de red
- ausencia de déficit estructural
- independencia de subsidios externos

---

10. Incentivo estructural (clave del diseño)

El modelo no premia:

- volumen arbitrario
- explotación del sistema

El modelo premia:

- **participación real en consenso**
- **procesamiento de estado**
- **validación correcta de PGDM v2 outputs**

---

11. Propiedad emergente del sistema

El NHIO Model transforma la red en:

un sistema de infraestructura computacional distribuida pagado por ejecución de eventos verificables

Esto implica:

- los nodos no son servidores pasivos
- son **actores económicos de la ejecución del juego**
- la red es simultáneamente:
    - motor de juego
    - sistema de liquidación
    - sistema de compensación

---

12. Relación con capas anteriores

12.1 HLCP (Hand Lifecycle Protocol)

→ genera eventos económicos (hands)

12.2 PGDM v2

→ asegura integridad del input (cartas)

12.3 LCCP Consensus Layer

→ valida estado

12.4 NHIO Model (este documento)

→ distribuye valor económico

---

13. Definición final del modelo

```
NHIO = {
   load_measurement,
   fee_generation,
   proportional_distribution,
   penalty_system,
   node_economic_balance
}
```

➜ Cierre del sistema conceptual

Con este módulo el sistema Crack Chain queda completo en 4 capas:

1. PGDM v2 → aleatoriedad determinística
2. HLCP → ejecución de manos
3. LCCP Consensus → validación distribuida
4. NHIO Model → economía de nodos

“Formal System Architecture Specification (Crack Chain Core Protocol)”

Formal System Architecture Specification (Crack Chain Core Protocol)

(Core Integration Layer — System-Wide Technical Specification)

---

1. Definición formal

El **Crack Chain Core Protocol (CCCP)** es la capa de arquitectura sistémica que integra todos los módulos previos en una sola estructura operativa coherente.

Su función es unificar:

- **PGDM v2** → generación determinística del deck
- **HLCP** → ciclo completo de la mano
- **LCCP Consensus Layer** → validación distribuida del estado
- **NHIO Model** → incentivos y operación económica de nodos

El resultado es un sistema de poker ejecutado como **protocolo distribuido verificable**, no como casino centralizado.

---

2. Objetivo del protocolo

El objetivo principal del CCCP es definir una arquitectura donde:

1. cada evento del juego sea reproducible,
2. cada transición de estado sea validable,
3. cada nodo tenga una función explícita,
4. el sistema completo sea auditável en tiempo real,
5. la operación no dependa de autoridad central.

---

3. Capas del sistema

El protocolo se divide en cuatro capas funcionales:

```
CCCP = {
   Layer_1: Deck_Determinism,
   Layer_2: Hand_Execution,
   Layer_3: Distributed_Consensus,
   Layer_4: Node_Incentive_And_Settlement
}
```

3.1 Layer 1 — Deck Determinism

Responsable de:

- construir el seed global
- derivar el mazo
- garantizar distribución reproducible
- eliminar RNG opaco

Este bloque está implementado por **PGDM v2**.

3.2 Layer 2 — Hand Execution

Responsable de:

- crear la instancia de mano
- ejecutar acciones de jugadores
- resolver apuestas
- llegar al showdown o cierre lógico

Este bloque está implementado por **HLCP**.

3.3 Layer 3 — Distributed Consensus

Responsable de:

- validar cada transición de estado
- aceptar o rechazar eventos
- sincronizar nodos anfitriones
- mantener coherencia de ledger

Este bloque corresponde a la capa de consenso LCCP.

3.4 Layer 4 — Node Incentive and Settlement

Responsable de:

- medir carga operativa
- distribuir fees
- penalizar nodos divergentes
- sostener económicamente la red

Este bloque corresponde al **NHIO Model**.

---

4. Principios arquitectónicos

4.1 Determinismo

Mismo input → mismo resultado.

Input_t = Input_t' => Output_t = Output_t'

4.2 Reproducibilidad

Cualquier nodo autorizado puede reconstruir el estado completo de una mano sin ambigüedad.

4.3 Distribución del control

No existe un servidor maestro único.
El control está repartido entre nodos anfitriones con validación cruzada.

4.4 Inmutabilidad lógica

Una vez aceptado un estado, cualquier alteración futura rompe la continuidad del hash chain.

---

5. Componentes del sistema

5.1 Host Node

Nodo operativo que:

- ejecuta lógica de mesa

- valida estado parcial
- participa en consenso
- registra eventos

5.2 Auditor Node

Nodo de verificación que:

- reconstruye manos
- compara hashes
- detecta divergencias
- genera evidencia de consistencia

5.3 Player Node

Nodo de interacción que:

- firma acciones
- envía decisiones de juego
- recibe estado actualizado

5.4 Settlement Engine

Motor que:

- liquida pot
- actualiza balances
- cierra mano
- emite bloque final de resultado

---

6. Modelo de comunicación

Toda comunicación interna del protocolo se basa en mensajes firmados y versionados.

6.1 Estructura mínima de mensaje

Message = {

```
  message_type,
  protocol_version,
  hand_id,
  sender_id,
  payload,
  signature,
  previous_hash
}
```

6.2 Reglas de transmisión

- cada mensaje debe estar firmado
- cada mensaje debe referenciar estado previo
- todo mensaje debe ser verificable por nodos externos
- mensajes fuera de secuencia se rechazan

---

7. Estructura del ledger

El sistema mantiene un ledger lógico por mano y un ledger agregado por mesa.

7.1 Hand Ledger

Contiene:

- seed
- deck
- acciones
- validaciones
- resultado final

7.2 Table Ledger

Contiene:

- historial de manos
- nodos activos
- estado de carga
- métricas de integridad

---

8. Flujo global del protocolo

El ciclo completo del sistema se define así:

Initialization
→ Seed Consensus
→ PGDM v2 Deck Generation
→ Hand Execution
→ Action Validation
→ Consensus Confirmation
→ Settlement
→ Finalization
→ Ledger Commit

---

9. Modelo de estado del sistema

El sistema puede expresarse como una máquina de estados jerárquica.

```
System_State_t = {
   network_state,
   table_state,
   hand_state,
   node_state,
   settlement_state
}
```

Cada subestado debe permanecer consistente con los demás.

---

10. Reglas de coherencia

Un sistema es válido si cumple simultáneamente:

```
Valid_System =
   Valid(Network_State)
   ∧ Valid(Table_State)
   ∧ Valid(Hand_State)
   ∧ Valid(Node_State)
   ∧ Valid(Settlement_State)
```

---

11. Regla de rechazo

Si existe inconsistencia en cualquiera de estos niveles:

- seed inválido
- deck no reproducible
- acción no firmada
- payout incoherente
- hash inconsistente

entonces:

- el estado se rechaza
- se conserva el último estado válido
- el nodo divergente es marcado

---

12. Manejo de fallo

12.1 Fallo local

Si un nodo falla localmente:

- se congela su estado parcial
- otro nodo puede reconstruir la transición
- la mano continúa si el consenso sigue válido

12.2 Fallo de consenso

Si el consenso no alcanza umbral:

- se pausa la mano
- se sincroniza el último bloque válido
- se reevalúa la ronda de validación

---

13. Umbral de validez

La red adopta una lógica de aceptación por mayoría cualificada.

Accept(State_t) if Validations ≥ 2/3 of Active Host Nodes

Esto protege contra:

- nodos aislados
- intentos de manipulación
- forks locales no autorizados

---

14. Integración económica

El protocolo también define cómo el valor económico entra al sistema y cómo se distribuye.

14.1 Entrada de valor

Cada mano genera un fee operativo.

14.2 Distribución

Ese fee alimenta:

- host nodes
- reserva de red
- capa de auditoría
- sostenibilidad del protocolo

---

15. Propiedad emergente

Crack Chain Core Protocol no actúa como una app de poker.

Actúa como:

un sistema operativo distribuido para juegos de poker verificables, con ejecución determinística, auditoría continua y compensación automática por infraestructura.

---

16. Definición final del Core Protocol

CCCP = {

```
  deterministic_deck_layer,
  hand_lifecycle_layer,
  distributed_consensus_layer,
  node_incentive_layer,
  settlement_and_audit_layer
}
```

---

17. Conclusión técnica

Con esta especificación, Crack Chain deja de ser una colección de ideas sueltas y pasa a ser un **stack protocolar completo**:

- **base criptográfica**
- **motor de mano**
- **consenso distribuido**
- **liquidación económica**
- **auditoría reproducible**

Es decir: un protocolo integral, no una sala de poker tradicional.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Protocol Versioning, Governance & Upgrade Rules”

Ahí se define cómo evoluciona el sistema sin romper compatibilidad ni integridad estructural.

Protocol Versioning, Governance & Upgrade Rules

(Crack Chain Core Protocol — Evolution Control Layer)

---

1. Definición formal

El **Protocol Versioning, Governance & Upgrade Rules** define cómo evoluciona Crack Chain sin romper:

- compatibilidad entre nodos,
- reproducibilidad del estado,
- integridad del ledger,
- validez histórica de manos previas.

Su función no es política en sentido abstracto, sino **operacional**: controlar cambios de protocolo con trazabilidad, seguridad y continuidad funcional.

---

2. Principio rector

Todo cambio del sistema debe cumplir esta regla:

New_Version must preserve:

historical_validity

deterministic_replay

consensus_integrity

node_compatibility_window

Si una actualización rompe cualquiera de esos puntos, no puede entrar directamente a producción.

---

3. Esquema de versionado

El protocolo usa versionado semántico estructurado:

CCCP = MAJOR.MINOR.PATCH

3.1 MAJOR

Cambios que afectan:

- lógica de consenso,
- reglas de mesa,
- estructura del deck,

- formato de mensaje,
- validación de estado.

Un cambio MAJOR puede requerir migración coordinada de nodos.

3.2 MINOR

Cambios compatibles hacia atrás que agregan:

- nuevas reglas opcionales,
- mejoras de rendimiento,
- optimizaciones de auditoría,
- extensiones funcionales no destructivas.

3.3 PATCH

Correcciones menores que no alteran:

- reglas base,
- formato de estado,
- compatibilidad del ledger,
- determinismo del protocolo.

---

4. Compatibilidad entre versiones

El sistema debe mantener una **ventana de coexistencia** entre versiones activas.

4.1 Regla de coexistencia

Version_n and Version_n-1 may coexist

if replay_consistency == true

and consensus_rule_set is not broken

Esto permite:

- actualización gradual de nodos,

- validación cruzada entre versiones,
- migración ordenada sin corte total.

4.2 Regla de rechazo

Una versión se rechaza si:

- no puede reconstruir estados anteriores,
- introduce ambigüedad en PGDM v2,
- rompe la lógica HLCP,
- altera el settlement final,
- modifica firmas o hashes incompatibles.

---

5. Modelo de governance

La governance de Crack Chain no se basa en votación política convencional, sino en **aceptación técnica de cambio**.

5.1 Unidad de gobierno

La unidad de decisión es la **Propuesta de Cambio de Protocolo**.

CCP = Protocol Change Proposal

Cada CCP debe especificar:

- objetivo del cambio,
- alcance técnico,
- módulos afectados,
- impacto en compatibilidad,
- prueba de integridad,
- plan de migración.

5.2 Actores de gobierno

5.2.1 Host Nodes

- evalúan impacto operativo,
- validan compatibilidad,
- ejecutan migración si procede.

5.2.2 Auditor Nodes

- verifican que el cambio no altera replay histórico,
- validan hashes y transiciones,
- certifican coherencia del upgrade.

5.2.3 Core Maintainers

- proponen cambios de especificación,
- redactan nuevas versiones,
- documentan reglas de transición.

---

6. Ciclo de propuesta

Una actualización sigue este flujo:

Draft → Review → Simulation → Validation → Activation → Monitoring

6.1 Draft

Se redacta el cambio propuesto con especificación formal.

6.2 Review

Los nodos anfitriones y auditores examinan el impacto.

6.3 Simulation

El cambio se prueba sobre un entorno de reproducción determinística.

6.4 Validation

Se verifica que:

- no rompe consenso,
- no altera manos históricas,

- no introduce ambigüedad.

6.5 Activation

Se activa la versión solo cuando existe umbral suficiente de compatibilidad.

6.6 Monitoring

Se observa comportamiento post-activación y se compara con el modelo esperado.

---

7. Regla de activación

Una nueva versión solo entra en vigencia si cumple:

Valid_Upgrade =

deterministic_replay_ok

∧ consensus_ok

∧ settlement_ok

∧ node_compatibility_ok

Si una condición falla, el sistema permanece en la versión anterior.

---

8. Gobernanza de cambios críticos

Hay cambios que no pueden hacerse de forma silenciosa.

8.1 Cambios críticos

Incluyen:

- modificación del algoritmo de reparto,
- cambio en reglas del mazo,
- alteración del umbral de consenso,
- cambio en estructura del message format,

- cambio en settlement logic.

Estos cambios requieren:

- notificación explícita,
- migración planificada,
- validación auditada,
- confirmación de nodos activos.

8.2 Cambios no críticos

Incluyen:

- refactor interno,
- mejoras visuales,
- optimización de logs,
- mejor manejo de errores,
- mejoras de performance no funcionales.

---

9. Entorno de prueba de versión

Todo upgrade debe pasar por un entorno determinístico de validación.

9.1 Requisitos del testnet interno

- reproducir manos históricas,
- simular consenso entre nodos,
- validar settlement final,
- comparar hashes con referencia.

9.2 Criterio de aprobación

Una versión pasa si:

- el replay produce el mismo resultado,
- el consenso converge,
- no hay divergencia de estado,
- el ledger permanece consistente.

---

10. Regla de rollback

Si una versión genera inconsistencia:

Rollback_to = last_valid_version

Condiciones de rollback:

- divergencia de hashes,
- fallo de consenso,
- incompatibilidad de nodo,
- error de settlement,
- ruptura del modelo PGDM v2.

El rollback debe ser reversible y auditable.

---

11. Registro de cambios

Toda actualización queda registrada en un changelog protocolar.

11.1 Campos mínimos

```
{
  version_id,
  timestamp,
  authoring_node,
  affected_modules,
  change_type,
  compatibility_status,
  audit_signature
}
```

Este registro no es decorativo: es parte del sistema de confianza.

---

12. Propiedad emergente de la governance

La governance del sistema no busca consenso político amplio, sino **estabilidad técnica verificable**.

Eso significa que el sistema prioriza:

- integridad,
- continuidad,
- trazabilidad,
- compatibilidad,
- determinismo.

No prioriza popularidad del cambio ni velocidad de publicación por encima de la seguridad estructural.

---

13. Definición final del módulo

```
Protocol_Versioning_Governance_Upgrade_Rules = {

  semver_scheme,

  compatibility_window,

  proposal_cycle,

  activation_rules,

  rollback_rules,

  changelog_registry

}
```

---

14. Conclusión técnica

Con este módulo, Crack Chain obtiene control de evolución sin perder su naturaleza protocolar.

En otras palabras:

- el sistema puede crecer,
- el sistema puede corregirse,
- el sistema puede migrar,
- pero no puede deformarse sin control.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Security, Threat Model & Adversarial Resistance”

Ahí se define cómo el protocolo resiste ataques, manipulación, colusión y divergencia de nodos.

Security, Threat Model & Adversarial Resistance

(Crack Chain Core Protocol — Defensive Integrity Layer)

---

1. Definición formal

El **Security, Threat Model & Adversarial Resistance** define el conjunto de supuestos, amenazas, controles y mecanismos de defensa que protegen a Crack Chain contra manipulación interna, ataques externos, nodos maliciosos y divergencia protocolar.

Su función es asegurar que el sistema permanezca:

- íntegro,
- verificable,
- determinístico,
- resistente a colusión,
- y operativo bajo condiciones adversas.

---

2. Principio rector de seguridad

La seguridad del sistema no depende de ocultamiento, sino de:

- reproducibilidad,
- consenso,
- trazabilidad,
- validación cruzada,
- y rechazo automático de estados inválidos.

Formalmente:

Security = Integrity + Determinism + Consensus + Auditability

---

3. Superficie de amenaza

El modelo de amenaza considera cinco vectores principales:

3.1 Amenaza interna

- nodo anfitrión malicioso
- auditor corrupto
- jugador que intenta alterar flujo de acciones
- operador que intenta introducir lógica no autorizada

3.2 Amenaza externa

- ataque de red
- suplantación de identidad
- replay fraud
- inyección de mensajes inválidos
- desincronización deliberada

3.3 Amenaza de consenso

- colusión entre nodos
- falsificación de hash
- estado divergente propuesto como válido
- intento de bifurcación local no autorizada

3.4 Amenaza de ejecución

- alteración del deck
- manipulación del action log
- discrepancia entre nodos sobre el mismo evento
- corrupción del settlement final

3.5 Amenaza económica

- abuso de fees
- spam de manos
- acaparamiento de carga operativa
- simulación de participación para obtener recompensa inmerecida

---

4. Modelo de adversario

Crack Chain asume que el adversario puede ser:

- **pasivo**, observando tráfico y estado,
- **activo**, intentando modificar mensajes o estados,
- **insider**, operando desde un nodo autorizado,
- **coordinado**, actuando en grupo,
- **económicamente oportunista**, explotando incentivos del protocolo.

El sistema no asume honestidad perfecta.
Asume **honestidad verificable**.

---

5. Controles de integridad

5.1 Firmas criptográficas

Toda acción relevante debe estar firmada.

```
Action = {

    payload,

    signature,

    signer_id

}
```

Cualquier mensaje sin firma válida se rechaza.

5.2 Hash chaining

Cada estado referencia el estado previo.

State_t = H(State_t-1 + Action_t + Consensus_t)

Esto impide:

- modificación silenciosa,
- inserción retroactiva,
- reordenamiento no autorizado,
- ocultamiento de eventos.

5.3 Determinismo reproducible

El estado debe poder reconstruirse por cualquier nodo válido.
Si la reconstrucción no coincide, el estado se considera inválido.

5.4 Validación cruzada

Los nodos no confían en un único reporte.

Se exige comparación entre múltiples nodos para confirmar:

- deck,
- acciones,
- balances,
- payout final,
- transición de estado.

---

6. Ataques considerados

6.1 Manipulación del deck

Intento de alterar orden o composición de cartas.

Defensa:

- PGDM v2 determinístico
- hash de seed
- validación cruzada del mazo
- rechazo de cualquier deck no reproducible

6.2 Falsificación de acción

Intento de enviar fold/call/raise/check no autorizado.

Defensa:

- firma digital
- control de secuencia
- validación de saldo
- verificación de turn ownership

6.3 Replay attack

Reutilización de mensajes previos como si fueran actuales.

Defensa:

- nonces de sesión
- hash de contexto
- previous_hash obligatorio
- rechazo por duplicidad

6.4 State divergence

Un nodo presenta un estado distinto al consenso.

Defensa:

- comparación de hashes

- majority validation
- aislamiento del nodo divergente
- rollback al último estado aceptado

6.5 Collusion attack

Grupo de nodos intenta acordar un estado falso.

Defensa:

- umbral de aceptación cualificada
- distribución de carga entre nodos
- auditoría independiente
- monitoreo de correlación anómala

6.6 Settlement manipulation

Intento de alterar el reparto final del pot.

Defensa:

- payout derivado del estado verificado
- settlement ligado al ledger
- doble validación de balances
- cierre con hash final

---

7. Mecanismos de resistencia adversarial

7.1 Rechazo por inconsistencia

Si un evento no coincide con el estado esperado, se invalida.

7.2 Aislamiento de nodo

Si un nodo falla repetidamente o introduce divergencia, se excluye del consenso activo.

7.3 Reproducción forense

Cualquier mano puede reconstruirse para auditoría completa.

7.4 Congelación de estado

Ante anomalía crítica, el sistema congela la transición antes de comprometer el ledger.

7.5 Rollback controlado

Si el error afecta coherencia global, se revierte al último bloque válido.

---

8. Clasificación de severidad

Severity = {

LOW: inconsistencia local no crítica,

MEDIUM: divergencia parcial de nodos,

HIGH: intento de manipulación del estado,

CRITICAL: ataque al consenso o al settlement

}

Respuesta por nivel:

- **LOW** → registro y observación
- **MEDIUM** → revalidación
- **HIGH** → aislamiento del nodo
- **CRITICAL** → rollback y reconfiguración del conjunto activo

---

9. Regla de tolerancia

El sistema tolera fallos parciales pero no tolera ambigüedad en el estado final.

Partial_Failure = acceptable

Final_State_Ambiguity = unacceptable

Esto significa que la red puede sobrevivir a nodos defectuosos, pero no puede aceptar dos resultados válidos para la misma mano.

---

10. 🔍 Monitoreo de seguridad

El protocolo exige monitoreo continuo de:

- latencia entre nodos
- divergencia de hash
- frecuencia de errores
- comportamiento estadístico anómalo
- concentración de validaciones entre los mismos nodos
- patrones sospechosos en acciones de jugadores

---

11. Propiedad emergente de seguridad

La seguridad de Crack Chain no es una capa externa añadida al sistema.
Es una consecuencia estructural de su arquitectura.

En otras palabras:

Si el estado no puede ser verificado, entonces no existe como estado válido.

Ese principio convierte la seguridad en una función del protocolo, no en un adorno operativo.

---

12. Definición final del módulo

Security_Threat_Model_Adversarial_Resistance = {

threat_surface,

adversary_model,

cryptographic_integrity,

```
  consensus_resistance,
  settlement_protection,
  rollback_and_isolation_rules
}
```

---

13. Conclusión técnica

Este módulo cierra la parte defensiva del Core Protocol.

Con él, Crack Chain queda especificado para resistir:

- corrupción de estado,
- manipulación del deck,
- falsificación de acciones,
- colusión entre nodos,
- y alteración del settlement.

La lógica es simple y estricta:

- lo verificable permanece,
- lo inconsistente se rechaza,
- lo divergente se aísla,
- lo no reproducible no entra al sistema.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Performance, Scalability & Load Distribution”

Ahí se define cómo la red crece sin perder determinismo, cómo distribuye carga entre nodos y cómo mantiene estabilidad bajo más mesas, más jugadores y más throughput.

Performance, Scalability & Load Distribution

(Crack Chain Core Protocol — Operational Scaling Layer)

---

1. Definición formal

La sección **Performance, Scalability & Load Distribution** define cómo Crack Chain mantiene ejecución estable, determinista y verificable mientras aumenta:

- el número de mesas,
- el número de jugadores,
- el volumen de acciones,
- la cantidad de nodos activos,
- y la carga de consenso en tiempo real.

Su función no es solo “hacer el sistema rápido”, sino asegurar que el crecimiento no destruya:

- la coherencia del estado,
- la reproducibilidad,
- la seguridad,
- ni la economía operativa de la red.

---

2. Principio rector

El sistema debe escalar sin perder determinismo.

Scale_Up must preserve:

determinism

consensus_integrity

replayability

settlement_consistency

Si el crecimiento introduce ambigüedad, el sistema se considera mal dimensionado.

---

3. Variables de carga

La carga operativa de la red depende de un conjunto de variables principales:

3.1 Carga por mesa

Cada mesa genera carga según:

- cantidad de manos por hora
- número de jugadores activos
- frecuencia de decisiones
- cantidad de validaciones necesarias

3.2 Carga por nodo

Cada nodo soporta:

- procesamiento de mensajes
- validación de eventos
- reproducción de estados
- participación en consenso
- sincronización de ledger

3.3 Carga global

La red completa carga por:

- mesas simultáneas
- fans de juego activos
- congestión de settlement
- actividad de auditoría
- tráfico de sincronización entre nodos

---

4. Modelo de carga funcional

La carga total se expresa como:

Load_total = f(Tables, Players, Actions, Validations, Audits, Settlements)

Y la carga por nodo como:

Load_node_i = f(Hands_i, Messages_i, Consensus_i, Audit_i)

---

5. Regla de distribución de carga

La red no debe depender de nodos sobrecargados.

La distribución debe seguir este principio:

$Workload_i \propto Capacity_i$

Donde la capacidad incluye:

- CPU disponible
- latencia operativa
- estabilidad de conexión
- historial de validación correcta
- participación previa en consenso

---

6. Balanceo dinámico

El sistema aplica balanceo dinámico para evitar cuellos de botella.

6.1 Distribución por mesas

Las mesas se asignan a nodos de forma que:

- ninguna mesa dependa de un solo punto de fallo,
- la validación pueda replicarse,
- el estado permanezca sincronizable.

6.2 Redistribución adaptativa

Si un nodo supera umbral de carga:

- se reduce su asignación
- se transfiere parte de la validación a nodos vecinos
- se preserva el hash chain
- se mantiene la continuidad de la mano

---

7. Umbral de saturación

Cada nodo tiene un límite operativo.

Node_Saturation = Load_node_i / Capacity_node_i

Estados operativos:

- **Normal**: carga dentro de rango esperado
- **Elevado**: carga alta pero tolerable
- **Crítico**: riesgo de retraso o divergencia
- **Saturado**: el nodo deja de recibir más carga nueva

---

8. Escalabilidad horizontal

Crack Chain escala agregando nodos, no inflando un nodo central.

Eso significa:

- más nodos anfitriones = más capacidad de mesas
- más auditores = más verificación paralela
- más distribución = menos fragilidad central

La arquitectura crece por **expansión lateral**, no por concentración vertical.

---

9. Escalabilidad vertical limitada

Se admite mejora interna de cada nodo, pero con límite estructural.

Se permite:

- mejor hardware
- mejor ancho de banda
- más eficiencia de validación
- mejor latencia

No se permite:

- convertir un nodo en autoridad central
- concentrar consenso en una sola máquina

- sustituir la red por un servidor maestro

---

10. Modelo de partición operativa

La red puede dividir el trabajo en particiones lógicas:

10.1 Partición por mesa

Cada mesa opera como una unidad de estado independiente.

10.2 Partición por región operativa

Nodos cercanos pueden compartir presión de validación.

10.3 Partición por rol

- Host nodes: ejecución
- Auditor nodes: verificación
- Settlement nodes: cierre de mano
- Reserve nodes: respaldo ante fallos

---

11. Métricas de performance

El protocolo debe vigilar, como mínimo, estas métricas:

- latencia de consenso
- tiempo de distribución de cartas
- tiempo de validación de acción
- tiempo de settlement
- tasa de reintento
- número de eventos por segundo
- número de mesas simultáneas
- tasa de divergencia de nodos

---

12. Regla de rendimiento aceptable

El sistema solo es funcional si mantiene:

Response_Time_table + Consensus_Time + Settlement_Time <= Max_Operational_Window

Si se excede esa ventana:

- se reduce carga,
- se redistribuye el trabajo,
- o se aplaza la nueva asignación de mesas.

---

13. Relación entre performance y seguridad

La velocidad no puede romper la integridad.

El protocolo prioriza:

1. determinismo
2. consenso
3. integridad
4. performance

Nunca al revés.

Una red rápida pero incoherente es inválida.
Una red más lenta pero verificable sigue siendo válida.

---

14. Relación con el modelo económico

La carga operativa debe reflejarse en el incentivo.

Si un nodo soporta más validación, más mesas o más tráfico:

- recibe más recompensa proporcional
- justifica mayor participación en fees
- mantiene alineación entre trabajo y pago

Esto evita que la red crezca sin corresponder el costo con la compensación.

---

15. Propiedad emergente del sistema

Crack Chain no escala como una app de poker convencional.
Escala como una **infraestructura distribuida de ejecución verificable**.

Eso implica:

- más actividad no rompe el sistema,
- la carga se reparte,
- el consenso se mantiene,
- el settlement sigue siendo reproducible.

---

16. Definición final del módulo

```
Performance_Scalability_Load_Distribution = {

    workload_metrics,

    dynamic_balancing,

    saturation_thresholds,

    horizontal_scaling,

    limited_vertical_scaling,

    partitioning_strategy,

    settlement_performance_rules

}
```

---

17. Conclusión técnica

Este módulo asegura que Crack Chain pueda crecer sin dejar de ser Crack Chain.

No se trata de “aguantar más tráfico” solamente.
Se trata de mantener intacta la lógica del protocolo bajo expansión real.

Si se cumple este módulo, la red puede absorber más mesas, más nodos y más actividad sin perder:

- exactitud,
- confianza protocolar,
- ni trazabilidad.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Interoperability, External Interfaces & API Surface”

Ahí se define cómo Crack Chain se conecta con wallets, nodos, clientes, sistemas de identidad y herramientas externas sin comprometer su núcleo determinista.

Interoperability, External Interfaces & API Surface

(Crack Chain Core Protocol — Integration Layer Specification)

---

1. Definición formal

La sección **Interoperability, External Interfaces & API Surface** define cómo Crack Chain expone capacidades controladas hacia el exterior sin comprometer su núcleo determinístico.

Su objetivo es permitir:

- conexión con clientes de juego,
- interacción con wallets,
- comunicación entre nodos,
- auditoría externa,
- integración con herramientas de administración,
- y eventual expansión hacia otros sistemas compatibles.

Todo interfaz externo debe preservar:

- integridad del estado,
- determinismo del protocolo,
- trazabilidad de eventos,
- y control de versiones.

---

2. Principio rector

La interoperabilidad no puede introducir ambigüedad.

External_Interface must preserve:

deterministic_core

state_integrity

message_order

version_compatibility

El núcleo no se adapta a la interfaz.
La interfaz se adapta al núcleo.

---

3. Alcance de la superficie API

La API superficial del sistema no expone lógica sensible sin control.

3.1 Se permite exponer

- autenticación de nodos
- estado de mesa
- estado de mano
- eventos de acción
- resultados de settlement
- historial verificable
- métricas operativas no sensibles

3.2 No se permite exponer

- seeds privados
- rutas internas de consenso
- material no firmado
- reglas ocultas de validación
- claves críticas
- lógica interna no destinada a reproducibilidad pública

---

4. Modelo de acceso

Toda interfaz debe estar asociada a un nivel de acceso.

4.1 Acceso público

Permite lectura limitada de:

- estado visible de mesas,
- disponibilidad,
- resultados publicados,
- eventos ya finalizados.

4.2 Acceso autenticado

Permite:

- participación en mesas,
- emisión de acciones firmadas,
- consulta de estado privado de sesión,
- interacción con wallet binding.

4.3 Acceso de nodo

Permite:

- validación,
- consenso,
- reproducción de estado,
- comunicación entre hosts,
- emisión de bloques de mano.

4.4 Acceso de auditoría

Permite:

- replay completo,
- verificación de hashes,
- comparación de ledger,
- análisis forense.

---

5. Modelo de interoperabilidad

Crack Chain puede interoperar en varios niveles:

5.1 Interoperabilidad interna

Entre módulos del propio sistema:

- PGDM v2
- HLCP
- LCCP Consensus
- NHIO Model
- settlement engine

5.2 Interoperabilidad externa controlada

Con sistemas como:

- wallets,
- interfaces cliente,
- dashboards,
- herramientas de auditoría,
- servicios de identidad.

5.3 Interoperabilidad futura

Con protocolos auxiliares siempre que:

- no rompan determinismo,
- no modifiquen el core,
- no alteren el ledger histórico.

---

6. Tipos de interfaces

6.1 Client Interface

Interfaz para el jugador o usuario final.

Funciones típicas:

- entrar a mesa
- firmar acción
- ver estado visible
- recibir actualización

- consultar resultado final

6.2 Host Interface

Interfaz para nodos anfitriones.

Funciones típicas:

- recibir eventos
- validar transiciones
- emitir consenso
- registrar estado
- publicar bloque de mano

6.3 Auditor Interface

Interfaz para nodos de auditoría.

Funciones típicas:

- reconstruir manos
- verificar hash chain
- comparar logs
- generar evidencia
- detectar divergencias

6.4 Settlement Interface

Interfaz para liquidación.

Funciones típicas:

- ejecutar payout
- cerrar balances
- emitir confirmación final
- registrar cierre de mano

---

7. API Surface mínima

La API mínima del sistema debe cubrir cuatro familias de funciones.

7.1 Identity APIs

- registrar nodo
- verificar firma
- enlazar wallet
- validar rol

7.2 Table APIs

- crear mesa
- unirse a mesa
- abandonar mesa
- consultar estado

7.3 Hand APIs

- iniciar mano
- enviar acción
- validar transición
- consultar log

7.4 Settlement APIs

- calcular payout
- cerrar mano
- emitir hash final
- consultar cierre

---

8. Regla de diseño de interfaces

Toda llamada API debe ser:

- idempotente cuando aplique,
- firmada cuando afecte estado,
- versionada,
- trazable,
- y validable por un nodo externo.

API_Call = {

```
  version,
  signer,
  timestamp_hash,
  payload,
  signature,
  replay_safe
}
```

---

9. Protocolo de mensajes externos

La comunicación externa usa mensajes estructurados.

9.1 Campos mínimos

- protocol_version
- message_type
- sender_id
- session_id
- hand_id
- payload
- signature
- previous_hash

9.2 Regla de aceptación

Un mensaje externo solo es válido si:

- pertenece a la versión correcta,
- está firmado,
- respeta secuencia,
- coincide con el contexto de sesión,
- no rompe el estado local.

---

10. Compatibilidad entre versiones externas

La capa de interfaces debe soportar transición controlada.

Regla básica

Una interfaz nueva puede coexistir con una anterior solo si ambas producen:

- mismo estado lógico,
- mismo efecto en el ledger,
- misma reconstrucción del evento.

Si el formato cambia, el sistema debe mantener un adaptador o una ventana de compatibilidad.

---

11. Adaptadores e integración

El protocolo permite adaptadores para:

- wallets compatibles,
- sistemas de autenticación,
- herramientas de monitoreo,
- frontends alternativos,
- módulos de reporting.

Todo adaptador debe actuar como capa de traducción, nunca como capa de decisión.

---

12. Reglas de no interferencia

La capa externa no puede:

- alterar reglas del mazo,
- alterar el orden de acciones,
- alterar el consenso,
- alterar el settlement,
- alterar el hash chain.

La interfaz transmite, pero no gobierna.

---

13. Criterio de integración aceptable

Una integración es válida si cumple estas condiciones:

Valid_Integration =

state_preserved

∧ signatures_verified

∧ replay_consistent

∧ version_compatible

∧ no_core_modification

---

14. Propiedad emergente de la superficie API

La API de Crack Chain no es una puerta de entrada al control del sistema.
Es una **superficie controlada de interacción verificable**.

Eso implica:

- el exterior puede leer,
- puede actuar dentro de reglas,
- puede auditar,
- pero no puede deformar el protocolo.

---

15. Definición final del módulo

Interoperability_External_Interfaces_API_Surface = {

access_model,

api_families,

```
  message_protocol,
  version_compatibility,
  adapter_rules,
  non_interference_constraints
}
```

---

16. Conclusión técnica

Con esta capa, Crack Chain puede conectarse con el mundo exterior sin perder su identidad protocolar.

La conclusión operativa es simple:

- lo externo se integra,
- lo interno permanece estable,
- lo verificable se conserva,
- lo incompatible se rechaza.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Data Model, Event Schema & Ledger Structure”

Ahí se define exactamente cómo se representan manos, acciones, nodos, balances, hashes y bloques dentro del sistema.

Data Model, Event Schema & Ledger Structure

(Crack Chain Core Protocol — Canonical Data Layer Specification)

---

1. Definición formal

La sección **Data Model, Event Schema & Ledger Structure** define cómo Crack Chain representa, organiza y preserva toda la información crítica del sistema.

Su función es establecer una estructura única, consistente y reproducible para:

- jugadores,
- nodos,
- mesas,
- manos,
- acciones,
- estados,
- settlements,
- y bloques de auditoría.

El objetivo es que cada dato tenga una forma canónica y verificable dentro del protocolo.

---

2. Principio rector

Todo dato que afecte al sistema debe ser:

- estructurado,
- versionado,
- firmado cuando corresponda,
- trazable,
- y reproducible.

Valid_Data = structured + signed + versioned + traceable + replayable

---

3. Modelo de datos base

El sistema se compone de entidades protocolarias principales.

3.1 Entidad Player

Representa a un participante humano o lógico del juego.

Campos mínimos:

```
Player = {
    player_id,
    wallet_id,
    role,
    status,
    balance,
    session_binding
}
```

3.2 Entidad Node

Representa un nodo de la red.

```
Node = {
    node_id,
    node_type,
    capacity,
    load,
    reputation,
    consensus_status
}
```

Tipos posibles:

- host node
- auditor node
- reserve node
- settlement node

3.3 Entidad Table

Representa una mesa activa o histórica.

```
Table = {
    table_id,
    ruleset_id,
    host_nodes,
    players,
    current_hand_id,
    table_state
}
```

3.4 Entidad Hand

Representa una mano individual.

```
Hand = {
    hand_id,
    table_id,
    seed_hash,
    deck_hash,
    action_log_hash,
    settlement_hash,
    final_status
}
```

---

4. Event Schema

Cada evento dentro del sistema debe seguir una estructura homogénea.

4.1 Definición general

```
Event = {
    event_id,
    protocol_version,
    event_type,
    actor_id,
    table_id,
    hand_id,
    timestamp_hash,
    payload,
    signature,
    previous_event_hash
}
```

4.2 Propiedades obligatorias

Todo evento debe incluir:

- identificador único,
- tipo de evento,
- actor que emite,
- contexto de mesa,
- contexto de mano,
- referencia al evento anterior,
- firma válida,
- y carga estructurada.

---

5. Tipos de eventos

5.1 Table Events

Eventos relacionados con la mesa.

Ejemplos:

- table_created
- table_opened
- table_paused
- table_closed
- table_rebalanced

5.2 Hand Events

Eventos relacionados con la mano.

Ejemplos:

- hand_initialized
- deck_generated
- cards_dealt
- action_received
- action_validated
- showdown_executed
- pot_settled
- hand_finalized

5.3 Node Events

Eventos relacionados con nodos.

Ejemplos:

- node_registered
- node_synced
- node_validated
- node_penalized
- node_isolated
- node_reintegrated

5.4 Settlement Events

Eventos relacionados con liquidación.

Ejemplos:

- payout_calculated
- balance_updated
- settlement_committed
- settlement_rejected

---

6. Regla de secuencialidad

Los eventos no pueden existir fuera de orden lógico.

Event_n must reference Event_n-1

Esto garantiza:

- encadenamiento correcto,
- no duplicidad,
- no inserción oculta,
- no reordenamiento arbitrario.

---

7. Ledger Structure

El ledger del sistema es la fuente canónica del estado histórico.

Se divide en tres niveles.

7.1 Hand Ledger

Contiene el historial de una mano específica.

Incluye:

- seed,

- deck,
- acciones,
- validaciones,
- resultado final,
- hash de cierre.

7.2 Table Ledger

Contiene el historial agregado de la mesa.

Incluye:

- manos jugadas,
- nodos activos,
- cambios de estado,
- comportamiento operativo,
- métricas agregadas.

7.3 Global Protocol Ledger

Contiene la historia estructural del sistema.

Incluye:

- versiones del protocolo,
- upgrades,
- governance events,
- penalizaciones de nodos,
- cambios de configuración global.

---

8. Modelo de hash chain

Cada bloque de datos debe enlazarse con el anterior.

Ledger_Block_t = H(Ledger_Block_t-1 + Event_Set_t + Consensus_t)

Esto produce:

- continuidad histórica,

- integridad criptográfica,
- detección inmediata de manipulación.

---

9. Estado canónico

En todo momento debe existir una única interpretación válida del estado.

Canonical_State = max(agreement among valid nodes)

Si dos nodos describen el mismo evento de forma distinta:

- se activa verificación,
- se compara against ledger,
- se rechaza la versión incoherente.

---

10. Serialización de datos

La serialización debe ser estable y predecible.

Requisitos:

- formato fijo,
- orden de campos definido,
- tipos explícitos,
- compatibilidad entre versiones,
- validación de integridad antes de parseo.

El sistema puede usar representaciones como:

- JSON estructurado,
- binary canonical encoding,
- protobuf-like schema,
- o formato protocolar propio.

Lo importante es que la serialización sea **determinista**.

---

11. Reproducibilidad del ledger

Cualquier nodo válido debe poder reconstruir el ledger a partir de:

- eventos firmados,
- hashes previos,
- reglas del protocolo,
- y versión aplicada.

Rebuilt_Ledger = Replay(Events, Ruleset, Version)

Si el replay no coincide con el ledger original, el estado se considera inválido.

---

12. Inmutabilidad lógica

Una vez que un bloque entra al ledger:

- no se edita,
- no se sobreescribe,
- no se reinterpreta sin dejar rastro.

Las correcciones solo pueden entrar como:

- nuevos eventos,
- reversión protocolar,
- o bloques compensatorios.

Nunca como edición silenciosa del pasado.

---

13. Validación de consistencia

Cada entrada debe pasar validaciones de:

- tipo,
- secuencia,
- firma,
- referencia previa,
- coherencia con el estado,
- compatibilidad de versión.

Valid_Record =

type_ok

∧ sequence_ok

∧ signature_ok

∧ previous_hash_ok

∧ state_consistent

∧ version_compatible

---

14. Definición final del módulo

Data_Model_Event_Schema_Ledger_Structure = {

canonical_entities,

event_schema,

sequential_integrity,

multi_layer_ledger,

hash_chain,

replayable_state

}

---

15. Conclusión técnica

Con este módulo, Crack Chain obtiene una gramática interna de datos.

Eso significa que el sistema deja de depender de interpretaciones ambiguas y pasa a operar con:

- entidades claras,

- eventos verificables,
- ledger reproducible,
- y estado canónico único.

En términos prácticos: este bloque convierte el protocolo en una máquina de información consistente, no en una simple aplicación transaccional.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Monitoring, Observability & Diagnostic Framework”

Ahí se define cómo el sistema se observa a sí mismo, cómo reporta anomalías y cómo produce evidencia operacional sin romper privacidad ni determinismo.

Monitoring, Observability & Diagnostic Framework

(Crack Chain Core Protocol — Runtime Inspection Layer)

---

1. Definición formal

La sección **Monitoring, Observability & Diagnostic Framework** define cómo Crack Chain observa su propio funcionamiento en tiempo real, detecta anomalías, conserva evidencia operativa y permite diagnóstico técnico sin comprometer el determinismo del protocolo.

Su objetivo es que el sistema pueda responder, con precisión, a estas preguntas:

- qué ocurrió,
- cuándo ocurrió,
- en qué nodo ocurrió,
- bajo qué versión ocurrió,
- qué estado produjo,
- y si ese estado fue válido o no.

---

2. Principio rector

El sistema debe ser observable sin volverse manipulable.

Observability must preserve:

protocol_integrity

state_determinism

audit_trail

privacy_constraints

La observabilidad no puede alterar la ejecución.
Solo puede medirla, registrarla y explicarla.

---

3. Objetivos del framework

El framework cumple cinco funciones esenciales:

1. **Monitoreo operativo**
    - medir carga
    - medir latencia
    - medir disponibilidad
    - medir rendimiento por nodo y por mesa
2. **Observabilidad estructural**
    - rastrear acciones
    - rastrear transiciones
    - rastrear settlement
    - rastrear consenso
3. **Diagnóstico**
    - identificar fallos
    - aislar nodos defectuosos
    - detectar divergencias
    - reconstruir cadenas de eventos
4. **Evidencia forense**
    - preservar historial
    - demostrar coherencia
    - soportar auditoría externa
5. **Prevención de degradación**

    - anticipar saturación
    - reducir riesgo de colapso
    - activar redistribución operativa

---

4. Dominios de observación

4.1 Nivel de red

Mide:

- conectividad entre nodos,
- latencia de sincronización,
- pérdida de mensajes,
- retrasos de consenso,
- disponibilidad de canales.

4.2 Nivel de mesa

Mide:

- estado de la mesa,
- actividad por mano,
- participación de jugadores,
- tiempos de acción,
- frecuencia de settlement.

4.3 Nivel de mano

Mide:

- secuencia de eventos,
- integridad del deck,
- validez de acciones,
- coherencia de transitions,
- cierre correcto.

4.4 Nivel de nodo

Mide:

- carga,
- reputación,

- errores,
- divergencias,
- participación en consenso,
- historial de validación.

4.5 Nivel de protocolo

Mide:

- compatibilidad de versión,
- salud de módulos,
- estabilidad de upgrades,
- comportamiento global del sistema.

---

5. Modelo de observabilidad

La observabilidad se estructura en tres capas:

```
Observability = {

    metrics,

    logs,

    traces

}
```

5.1 Metrics

Valores cuantificables del sistema.

Ejemplos:

- manos por minuto,
- tiempo medio de consenso,
- tasa de error por nodo,
- latencia promedio de settlement,
- porcentaje de carga por host.

5.2 Logs

Registros estructurados de eventos.

Ejemplos:

- acción recibida,
- hash validado,
- nodo penalizado,
- transición aceptada,
- mano finalizada.

5.3 Traces

Trazas completas de una mano o sesión.

Permiten reconstruir:

- cadena exacta de eventos,
- orden de ejecución,
- rutas entre nodos,
- puntos de decisión,
- causas de fallo.

---

6. Registro observacional

Todo dato de observabilidad debe ser:

- timestamped,
- versioned,
- context-bound,
- immutable,
- and replayable.

Observation = {

obs_id,

protocol_version,

scope,

source_id,

```
  context_id,
  value,
  timestamp_hash,
  signature
}
```

---

7. Observabilidad sin interferencia

La observabilidad no puede:

- modificar acciones,
- alterar el deck,
- influir en consenso,
- condicionar settlement,
- ni exponer material sensible no autorizado.

Por diseño, el sistema observa desde fuera del plano decisional.

---

8. Diagnóstico técnico

El diagnóstico se activa cuando una observación excede umbral tolerable.

8.1 Síntomas observables

- latencia inusual
- desincronización de nodos
- discrepancia entre hashes
- retraso en settlement
- divergencia en deck reconstruction
- error de firma
- caída de disponibilidad

8.2 Diagnóstico del origen

El sistema intenta determinar si la anomalía se originó en:

- red,
- nodo,
- mesa,
- hand lifecycle,
- consenso,
- settlement,
- o versión de protocolo.

---

9. Umbrales de alarma

Cada métrica posee umbral operativo.

```
Threshold = {
    warning,
    degraded,
    critical
}
```

9.1 Warning

Condición todavía tolerable, pero fuera del rango esperado.

9.2 Degraded

La operación sigue, pero con riesgo de acumulación de error.

9.3 Critical

El sistema debe intervenir de inmediato para evitar corrupción o pérdida de consistencia.

---

10. Proceso de detección

El ciclo básico es:

Observe → Compare → Classify → Isolate → Revalidate → Report

Paso a paso:

- observar datos
- compararlos con el estado esperado
- clasificar la anomalía
- aislar el origen
- revalidar el estado
- reportar el diagnóstico

---

11. Aislamiento diagnóstico

Si un nodo o una mesa presenta falla, el sistema debe poder aislar sin destruir la red completa.

11.1 Aislamiento local

Afecta solo a la mesa o al nodo defectuoso.

11.2 Aislamiento de consenso

Retira temporalmente del conjunto válido a nodos divergentes.

11.3 Aislamiento preventivo

Se activa antes de que el fallo se convierta en inconsistencia global.

---

12. Evidencia forense

El framework debe conservar evidencia suficiente para reconstruir cualquier incidente.

Incluye:

- logs de acciones,
- hashes de mano,
- validaciones de consenso,
- settlement final,

- versión aplicada,
- nodo responsable,
- secuencia de eventos.

Esta evidencia permite determinar no solo **qué pasó**, sino **si lo que pasó era válido**.

---

13. Propiedad emergente de la observabilidad

Crack Chain no necesita adivinar su estado.

El sistema lo sabe, porque:

- registra,
- compara,
- valida,
- y conserva evidencia.

Eso convierte la observabilidad en una función estructural del protocolo, no en una herramienta periférica.

---

14. Definición final del módulo

```
Monitoring_Observability_Diagnostic_Framework = {

    metrics_layer,

    logs_layer,

    traces_layer,

    anomaly_detection,

    diagnostic_classification,

    node_isolation,

    forensic_evidence

}
```

---

15. Conclusión técnica

Este módulo completa la capacidad del sistema para verse a sí mismo en operación real.

Con él, Crack Chain puede:

- detectar degradación,
- explicar fallos,
- aislar divergencias,
- y sostener auditoría técnica completa.

La regla central queda intacta:

Si el sistema no puede observarse con precisión, entonces no puede gobernarse con seguridad.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Deployment, Bootstrapping & Operational Lifecycle”

Ahí se define cómo la red se despliega, cómo arranca desde cero, cómo se incorporan nodos y cómo se mantiene la operación continua.

Deployment, Bootstrapping & Operational Lifecycle

(Crack Chain Core Protocol — System Initialization and Runtime Continuity Layer)

---

1. Definición formal

La sección **Deployment, Bootstrapping & Operational Lifecycle** define cómo Crack Chain se despliega, cómo arranca desde un estado vacío, cómo incorpora nodos activos y cómo mantiene continuidad operativa durante todo su ciclo de vida.

Su objetivo es asegurar que el sistema pueda pasar de:

- **cero infraestructura activa**
- a **red funcional validada**
- sin perder determinismo, integridad ni control de versión.

---

2. Principio rector

El arranque del sistema debe ser reproducible y verificable.

Deployment must preserve:

```
deterministic_initialization

version_control

consensus_readiness

auditability
```

No existe despliegue válido si el estado inicial no puede ser reconstruido.

---

3. Fases del ciclo operativo

El ciclo de vida operacional del sistema se divide en seis fases:

```
Lifecycle = {

    pre_deployment,

    bootstrap,

    activation,

    steady_state,

    maintenance,

    retirement

}
```

3.1 Pre-Deployment

Fase previa a la activación real del protocolo.

Incluye:

- verificación de versión,
- validación de configuración,
- comprobación de nodos candidatos,
- revisión de reglas de consenso,
- chequeo de compatibilidad de interfaces.

En esta fase todavía no se acepta tráfico productivo.

3.2 Bootstrap

Fase de inicio estructural de la red.

Incluye:

- generación del estado inicial,
- registro de nodos fundadores,
- sincronización de parámetros globales,
- creación de ledger base,
- activación del primer conjunto de reglas.

El bootstrap debe producir un estado inicial único y reproducible.

3.3 Activation

Fase en la que la red queda habilitada para operar.

Incluye:

- apertura de mesas,
- enrolamiento de jugadores,
- habilitación del ciclo HLCP,
- emisión de semillas iniciales,
- validación del primer consenso.

3.4 Steady State

Fase de operación estable.

Incluye:

- manos activas,
- consenso continuo,
- settlement recurrente,
- monitoreo de nodos,
- distribución de carga,
- auditoría permanente.

3.5 Maintenance

Fase de mantenimiento controlado.

Incluye:

- upgrades,
- rotación de nodos,
- parches,
- optimización de performance,
- correcciones de estado,
- validaciones post-cambio.

3.6 Retirement

Fase de cierre de una instancia o versión.

Incluye:

- congelación de nuevas manos,
- finalización de estados pendientes,
- exportación del ledger,
- cierre de sesiones,
- preservación histórica.

---

4. Bootstrap model

El bootstrap del sistema no es un inicio arbitrario.
Es una construcción controlada de estado canónico.

4.1 Inputs mínimos

- versión del protocolo
- conjunto inicial de nodos
- parámetros de reglas
- configuración de consenso
- parámetros de settlement
- configuración de seguridad
- schema de datos base

4.2 Output mínimo

```
Bootstrap_Output = {

    genesis_state,

    protocol_version,

    node_registry,

    consensus_parameters,

    ledger_root_hash

}
```

---

5. Genesis state

El **genesis state** es el estado fundacional del sistema.

Debe contener:

- identidad del protocolo,
- versión base,
- reglas iniciales,
- configuración de nodos fundadores,
- hash raíz del ledger.

No puede generarse de forma distinta en dos despliegues equivalentes sin romper reproducibilidad.

---

6. Registro de nodos

Antes de activarse, cada nodo debe ser registrado.

6.1 Campos mínimos del nodo

```
Node_Record = {
    node_id,
    node_type,
    role,
    capacity,
    signature_key,
    software_version,
    status
}
```

6.2 Estados del nodo

- pending
- verified
- active
- degraded
- isolated
- retired

---

7. Secuencia de arranque

El arranque formal sigue esta cadena:

Initialize → Verify → Register → Sync → Commit → Activate

7.1 Initialize

Se cargan reglas, parámetros y estructura base.

7.2 Verify

Se valida que el binario, la versión y la configuración sean compatibles.

7.3 Register

Se incorporan nodos permitidos al registro activo.

7.4 Sync

Se sincroniza seed, ledger root y parámetros globales.

7.5 Commit

Se confirma el estado inicial como base de operación.

7.6 Activate

La red queda habilitada para manos y mesas productivas.

---

8. Modelo de despliegue

El sistema puede desplegarse en distintos modos, siempre que no se rompa el núcleo protocolar.

8.1 Deployment local

Usado para pruebas, simulación y validación interna.

8.2 Deployment federado

Usado cuando varios nodos cooperan bajo una configuración común.

8.3 Deployment distribuido

Usado en red operativa real con múltiples nodos anfitriones y auditores.

8.4 Deployment staged

Usado para migraciones parciales y activación gradual de versiones.

---

9. Operative lifecycle

Una vez activado, el sistema entra en vida operativa continua.

9.1 Operative loop

Hand_Execution → Consensus_Check → Settlement → Logging → Monitoring → Next_Hand

9.2 Propiedad del ciclo

Cada ciclo debe:

- mantener estado consistente,
- conservar historial,
- producir hash verificable,
- ser recuperable por replay.

---

10. 🛠️ Maintenance lifecycle

El mantenimiento no puede alterar la historia del sistema.

10.1 Allowed maintenance

- actualización de nodos,
- ajuste de rendimiento,
- corrección de errores,
- rotación de llaves,
- reequilibrio de carga.

10.2 Forbidden maintenance

- reescritura silenciosa del ledger,
- cambio no versionado de reglas,
- alteración oculta del deck,
- modificación retroactiva de settlement.

---

11. Node lifecycle

Cada nodo sigue su propio ciclo dentro del sistema.

```
Node_Lifecycle = {

    onboarded,

    verified,

    active,

    monitored,

    degraded,

    isolated,

    reintegrated_or_retired

}
```

11.1 Onboarded

El nodo es propuesto para ingreso.

11.2 Verified

El nodo pasa controles técnicos y de identidad.

11.3 Active

El nodo participa en consenso y ejecución.

11.4 Monitored

El nodo es observado continuamente.

11.5 Degraded

El nodo presenta riesgo operativo.

11.6 Isolated

El nodo es retirado temporalmente del consenso.

11.7 Reintegrated or retired

El nodo vuelve a operar o se retira de forma controlada.

---

12. Recovery model

Si el sistema se interrumpe, debe poder volver a estado coherente.

12.1 Recovery from snapshot

Se restaura desde un estado validado previamente.

12.2 Recovery from ledger

Se reconstruye el estado por replay de eventos firmados.

12.3 Recovery from consensus

Se toma como verdad el último estado de consenso cualificado.

---

13. Reglas de continuidad operativa

La operación continua exige que:

- no exista ambigüedad en el estado de mano,
- no exista doble settlement,
- no exista fork no autorizado,
- no exista nodo activo sin versión compatible,
- no exista mesa sin respaldo lógico.

Operational_Continuity =

  valid_state

$\wedge$ valid_consensus

$\wedge$ valid_settlement

$\wedge$ valid_versioning

$\wedge$ valid_node_set

---

14. Shutdown protocol

El cierre del sistema debe ser ordenado.

14.1 Controlled shutdown

- congelar nuevas manos
- finalizar mano activa
- cerrar mesa
- exportar ledger
- preservar logs
- marcar nodos en salida

14.2 Emergency shutdown

Solo se usa ante corrupción crítica.

- congelación inmediata
- bloqueo de nuevas acciones
- preservación de evidencia
- rollback al último bloque válido

---

15. Propiedad emergente del ciclo operativo

Crack Chain no se “enciende” como una app.

Se **instancia**, se **verifica**, se **activa**, se **observa** y se **mantiene** como infraestructura protocolar.

Eso significa que el ciclo de vida del sistema es parte del protocolo, no una tarea externa.

---

16. Definición final del módulo

```
Deployment_Bootstrapping_Operational_Lifecycle = {
    pre_deployment_checks,
    genesis_bootstrap,
    node_registration,
    activation_sequence,
    steady_state_operation,
    maintenance_rules,
    recovery_paths,
    shutdown_protocol
}
```

---

17. Conclusión técnica

Con este bloque, Crack Chain obtiene una definición completa de inicio, operación y cierre.

La red ya no depende solo de reglas de juego o de consenso.
También tiene formalizado:

- cómo nace,
- cómo se mantiene,
- cómo se recupera,
- y cómo se apaga sin perder integridad.

➜ Siguiente bloque natural

El siguiente apartado lógico es:

“Appendix: Formal Definitions, Notation & Terminology”

Ahí se consolidan todas las definiciones, abreviaturas, símbolos y términos usados en el protocolo para dejar el documento completamente normativo.

Appendix: Formal Definitions, Notation & Terminology

(Crack Chain Core Protocol — Canonical Specification Layer)

---

1. Definición del apéndice

Este apéndice establece el **glosario formal, notación matemática, definiciones canónicas y terminología operacional** utilizada en todo el protocolo Crack Chain.

Su función es eliminar ambigüedad semántica y garantizar que cualquier implementación interprete el sistema de forma idéntica.

---

2. Principio de interpretación

All terms MUST be interpreted:

context-free

version-bound

deterministically

schema-consistent

Si una definición entra en conflicto con ejecución, prevalece la especificación estructural del protocolo sobre lenguaje natural.

---

3. Notación general

3.1 Estructuras

- `{}` → objeto estructurado
- `[]` → lista ordenada
- `()` → agrupación lógica
- `→` → transformación o resultado
- `∧` → conjunción lógica (AND)
- `∨` → disyunción lógica (OR)

3.2 Funciones

- `H(x)` → función hash criptográfica de x
- `f(x)` → función determinística del sistema
- `Consensus(x)` → resultado de validación distribuida
- `Transition(S, A)` → cambio de estado S bajo acción A
- `Replay(L)` → reconstrucción de estado desde ledger L

3.3 Identificadores

- `id` → identificador único de entidad
- `hash` → representación criptográfica del estado
- `signature` → prueba criptográfica de autoría
- `timestamp_hash` → marca temporal determinística

---

4. Definiciones canónicas del sistema

4.1 State (Estado)

State_t = representation of system at time t

Incluye:

- jugadores

- nodos
- mesa
- acciones
- deck
- resultados parciales

4.2 Event (Evento)

Event = atomic unit of state transition

Un evento es inmutable y encadenado al anterior mediante hash.

4.3 Node (Nodo)

Node = computational participant in consensus network

Puede ser:

- host
- auditor
- settlement
- reserve

4.4 Hand (Mano)

Hand = complete execution cycle of a poker game instance

Incluye:

- seed generation
- deck creation
- action loop
- consensus validation
- settlement

4.5 Ledger

Ledger = ordered sequence of immutable events

Propiedades:

- append-only
- hash-chained
- replayable
- verifiable

4.6 Seed

Seed = deterministic entropy source derived from system state

No es aleatorio externo.
Es función del estado del sistema.

4.7 Deck

Deck = deterministic permutation of base card set

Generado exclusivamente por PGDM v2.

4.8 Consensus

Consensus = majority validation of state among active nodes

Regla base:

Valid if valid_votes ≥ 2/3 active_nodes

4.9 Settlement

Settlement = final deterministic distribution of pot based on validated state

No puede alterarse después de hash final.

---

5. Reglas de identidad

5.1 Unicidad

Cada entidad debe tener un identificador único global.

$\forall$ entity $\exists!$ id

5.2 Inmutabilidad

Una vez emitido:

- event
- state hash
- ledger block

no puede modificarse sin generar nuevo evento.

5.3 Reproducibilidad

Replay(system_state) must equal original_state

---

6. Funciones críticas del sistema

Función	Definición
`H(x)`	hash criptográfico
`Consensus(x)`	validación distribuida
`Transition(S,A)`	evolución de estado
`PGDM_v2()`	generación determinística de deck
`HLCP()`	ciclo completo de mano
`NHIO()`	modelo económico de nodos

---

7. Tipos de estado

7.1 Initial State

Estado de inicio del sistema o mesa.

7.2 Intermediate State

Estado durante ejecución de la mano.

7.3 Final State

Estado cerrado tras settlement.

7.4 Invalid State

Estado rechazado por inconsistencia o fallo de consenso.

---

8. Tipos de validación

8.1 Structural validation

Verifica formato de datos.

8.2 Cryptographic validation

Verifica firmas y hashes.

8.3 Consensus validation

Verifica acuerdo entre nodos.

8.4 Deterministic validation

Verifica reproducibilidad del estado.

---

9. Semántica de ejecución

El sistema siempre opera bajo este modelo:

System = f(initial_state, events, consensus_rules)

No existe ejecución no determinista válida.

---

10. Jerarquía de verdad del sistema

Si hay conflicto entre interpretaciones:

1. Ledger hash chain (máxima prioridad)
2. Estado reproducido por consenso
3. Reglas del protocolo (CCCP)
4. Definición del módulo específico
5. Descripción textual

---

11. Terminología crítica

Deterministic

Resultado completamente predecible desde inputs.

Immutable

No modificable sin romper integridad del sistema.

Verifiable

Puede ser comprobado por cualquier nodo externo válido.

Reproducible

Puede ser reconstruido desde cero.

Consensus-bound

Dependiente de validación distribuida.

---

12. Regla de no ambigüedad

```
If term is ambiguous:
    use protocol definition
else:
    reject interpretation
```

---

13. Conclusión del apéndice

Este apéndice consolida Crack Chain como un sistema completamente formalizable:

- sin ambigüedad semántica
- sin interpretación libre del estado
- con reglas deterministas estrictas
- con estructura reproducible universal

---

14. Cierre del documento completo

Con este bloque, el **Crack Chain Core Protocol** queda completamente definido como:

un sistema computacional distribuido determinístico para ejecución, validación y liquidación de juegos de poker bajo consenso verificable.